3.4.3 Testing REST controllers with @WebMvcTest

Some integration tests might not need a fully initialized application context. For example, there's no need to load the web components when you're testing the data persistence layer. If you're testing the web components, you don't need to load the data persistence layer.

Spring Boot allows you to use contexts initialized only with a subgroup of components (beans), targeting a specific application slice. *Slice tests* don't use the `@SpringBootTest` annotation, but one of a set of annotations dedicated to particular parts of an application: Web MVC, Web Flux, REST client, JDBC, JPA, Mongo, Redis, JSON, and others. Each of those annotations initializes an application context, filtering out all the beans outside that slice.

We can test that Spring MVC controllers work as intended by using the `@WebMvcTest` annotation, which loads a Spring application context in a mock web environment (no running server), configures the Spring MVC infrastructure, and includes only the beans used by the MVC layer, like `@RestController` and `@RestControllerAdvice`. It's also a good idea to limit the context to the beans used by the specific controller under test. We can do so by providing the controller class as an argument to the `@WebMvcTest` annotation in a new `BookControllerMvcTests` class.

Listing 3.18 Integration tests for the web MVC slice

```
package com.polarbookshop.catalogservice.web;

import com.polarbookshop.catalogservice.domain.BookNotFoundException;
import com.polarbookshop.catalogservice.domain.BookService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
```